SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: ITA 06 COURSE NAME: MACHINE LEARNING COURSE

OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 20%

QUESTIONS: 15 Total Marks:100 Annexure A

SCENARIO-BASED QUESTIONS

Code of Conduct:

*I, **Bandaru Jayasri-192424301** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	*Conceptual Understan din g*	*Accura c y of Answer s*	*Applicati o n of Knowled ge*	*Completen es s of Response*	*Clarity & Presentati o n*	*Tota l*
Weightag e	*30%*	*20%*	*20%*	*15%*	*15%*	
Q1						
Q2						
Q3						

Signature of the student: Total Marks Awarded B.Jayasri

Annexure A

Short Answer Test

(5 Marks)

11. With the help of a neat diagram, explain the architecture of an **Artificial Neural Network (ANN)**. Describe the roles of input, hidden, and output layers, and explain how weights and biases are used in computation. Illustrate how forward propagation works with a simple example, and discuss how overfitting can be controlled using techniques such as dropout and regularization. (5 Marks)
12. Design a machine learning system for **weather forecasting** to predict rainfall

using features such as temperature, humidity, wind speed, and atmospheric pressure. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time predictions. (5 Marks)

13. In the context of **Recurrent Neural Networks (RNNs)**, explain the architecture and how they handle sequential data. Describe the concept of hidden states and how information is passed through time steps. Illustrate with an example such as text prediction or time-series forecasting, and discuss challenges like vanishing gradients and possible solutions. (5 Marks)

(5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary Marks)	
Conceptual Understanding	**30**	Demonstrate clear and cor understandin the concept	
			gaps
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly with r err
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concept minor mistake
Completeness of Response	**15**	Covers all required parts of the question	Covers parts minor omissio
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Underst e with issues

11. Artificial Neural Network (ANN) Architecture

ANN Architecture

```
Input Layer Hidden Layer Output Layer x1 ● --------\
 \ ● h1 --------\
 x2 ● ---------->-------- ● h2 ---------> ● Output (Y)
/ ● h3 --------/
 x3 ● --------/
```

Components of ANN

1. Input Layer

- Receives the input features.
- Example: Age, Income, Temperature, Humidity.
- It only passes data to the next layer.

2. Hidden Layer

- Performs mathematical computations.
- Learns patterns from the input data.
- Uses activation functions such as ReLU or Sigmoid.

3. Output Layer

- Produces the final prediction.
- Example:
  - 0 or 1 (Classification)
  - Predicted price (Regression)

Role of Weights and Biases

The neuron computes:

Z=∑(wixi)+bZ = \sum (w_i x_i) + bZ=∑(wixi)+b

where:

- xix_ixi = Inputs
- wiw_iwi = Weights
- bbb = Bias

The output is:
Y=f(Z)Y = f(Z)Y=f(Z)

where fff is the activation function.

Forward Propagation Example

Suppose:

- Input = 2
- Weight = 0.5
- Bias = 0.3

Calculation:

Z=(2×0.5)+0.3=1.3Z = (2 \times 0.5) + 0.3 = 1.3Z=(2×0.5)+0.3=1.3

Using the Step Activation Function:

Since 1.3>01.3 > 01.3>0,

Output = **1**

Thus, the ANN predicts the positive class.

Controlling Overfitting

1. Dropout

- Randomly disables some neurons during training.
- Prevents dependence on specific neurons.
- Improves generalization.

2. Regularization (L1/L2)

- Adds a penalty to large weights.
- Reduces model complexity.
- Prevents overfitting.

Conclusion

An Artificial Neural Network consists of input, hidden, and output layers connected by weights and biases. Forward propagation computes predictions, while techniques such as dropout and regularization improve the model's ability to generalize to new data.

12. Machine Learning System for Weather Forecasting

Problem Statement

Design a machine learning system to predict rainfall using:

- Temperature
- Humidity
- Wind Speed
- Atmospheric Pressure

System Architecture

```
Weather Dataset
 │
 ▼
Data Preprocessing
 │
 ▼
Feature Engineering
 │
 ▼
Machine Learning Model
 │
 ▼
Model Validation
 │
```

```
 ▼
Rainfall Prediction
 |
 ▼
Real-Time Deployment
```

Choice of Algorithm

Random Forest Classifier

Justification

- Handles nonlinear relationships.
- Works well with weather data.
- Reduces overfitting using multiple decision trees.
- Provides high prediction accuracy.

Data Preprocessing

- Remove missing values.
- Remove duplicate records.
- Normalize numerical features.
- Encode categorical variables if present.

Feature Engineering

Create useful features such as:

- Average temperature
- Humidity index
- Pressure difference
- Wind intensity

These improve prediction performance.

Model Training

- Split dataset:
    - 80% Training
    - 20% Testing
- Train the Random Forest model.
- Optimize hyperparameters if necessary.

Model Validation

Evaluate using:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Deployment

Deploy the trained model as:

- Web Application (Flask/Django)
- Mobile Application
- Cloud API

The system receives real-time weather data and predicts whether rainfall is expected.

Conclusion

A Random Forest-based weather forecasting system, combined with preprocessing, feature engineering, validation, and cloud deployment, provides accurate and reliable real-time rainfall predictions.

13. Recurrent Neural Network (RNN)

RNN Architecture

```
Time t1 Time t2 Time t3

x1 ─►[RNN]──► h1
 │
 ▼
x2 ─►[RNN]──► h2
 │
 ▼
x3 ─►[RNN]──► h3
 │
 ▼
 Output
```

What is an RNN?

A Recurrent Neural Network (RNN) is a neural network designed for **sequential data**. Unlike traditional neural networks, it remembers information from previous time steps using a hidden state.

Applications include:

- Text prediction
- Speech recognition
- Language translation
- Time-series forecasting

Hidden State

At each time step:

- Current input is received.
- Previous hidden state is combined with the current input.

- A new hidden state is generated.
- The hidden state carries information to the next time step.

This allows the RNN to remember previous information.

Example: Text Prediction

Input sequence:

```
"I love machine"
```

The RNN predicts the next word:

```
"I love machine learning"
```

The prediction is possible because the hidden state stores context from previous words.

Challenges

Vanishing Gradient

- Gradients become very small during backpropagation.
- The network struggles to learn long-term dependencies.

Exploding Gradient

- Gradients become excessively large.
- Training becomes unstable.

Solutions

Long Short-Term Memory (LSTM)

- Uses memory cells and gates.
- Retains important information over long sequences.
- Solves the vanishing gradient problem.

Gated Recurrent Unit (GRU)

- Simpler than LSTM.
- Requires fewer parameters.
- Learns long-term dependencies effectively.

Gradient Clipping

- Limits excessively large gradients.
- Prevents unstable training.

Conclusion

Recurrent Neural Networks process sequential data by maintaining hidden states across time steps. They are widely used in text prediction and time-series forecasting. Challenges such as vanishing gradients can be addressed using LSTM, GRU, and gradient clipping, leading to improved performance on long sequences.